SF 1 Pilot Findings and Report Implications

Nov 10, 2026 · @Mohamed khalil Ankri

Summary

At SF 1, ClickHouse answers all four queries 3.8 to 6.7 times faster than PostgreSQL in warm runs (design B, 30 runs each), with every 95% confidence interval above 1. This is consistent with H1 at the first of four sizes; H2a and H2b need the larger sizes and cannot be assessed yet.

Before any timing, the setup was validated: identical data in both engines and both designs, identical query results matching the TPC-H reference answers (after one fix to Q14), and a verified physical order for design D. The storage counters then confirmed how much each engine reads: about 8 to 12 times less data for ClickHouse, because it reads only the needed columns and compresses them.

The measurements also produced five findings that revise the M1 report: the host is WSL2 rather than native Linux; the PostgreSQL heap at SF 10 will be about 7.7 GB, not 9 GB; variability requires 30 warm runs, not 10; Q1, not Q6 or Q14, shows the largest speedup; and ClickHouse skips data for Q12 under design D, which the report classified as a non-skipping query.

Experimental setup as implemented

The setup matches the report on resources and settings, but the host is a Windows laptop running Linux through WSL2, not native Linux.

Component	Report (M1)	Implemented
Host	Native Linux, 8 logical CPUs, 16 GB RAM, SSD	Windows laptop, WSL2 (Ubuntu 22.04.3, kernel 6.18), Intel i5-10300H (4 cores, 8 threads), 16 GB RAM, SSD (E:)
Memory visible to Linux	16 GB	12 GB (set in `.wslconfig`), swap disabled
cgroup	v2	v2, systemd driver (forced with `cgroup_no_v1=all`)
Container runtime	Docker	Docker Engine 29.9.3 inside Ubuntu (not Docker Desktop)
PostgreSQL	18.X	18.6, pinned by image digest

Component	Report (M1)	Implemented
ClickHouse	26.8.X LTS	26.8.36.3, pinned by image digest
CPU and memory per container	4 CPUs, 8 GB	`cpuset 0-3` (2 physical cores with their 2 hyperthreads), 8 GB, no swap
Engine settings	Table 5	All Table 5 values verified inside both engines, including `use_query_condition_cache = 0`
Data generator	DBGen	tpch-kit DBGen 2.17.3, commit recorded in `env/dbgen.txt`

All settings were read back from the running engines (`SHOW` in PostgreSQL, `system.settings` in ClickHouse) and the container limits from the cgroup files, so the values above are verified, not assumed. PostgreSQL 18.6 runs with `io_method = worker` (the default).

Data loading: identical data in both engines

Both engines store exactly the same data under both designs. Each table was checked with a fingerprint: its row count and the sums of two decimal columns. A missing, duplicated or misparsed row would change at least one of these values.

Table	PostgreSQL	ClickHouse
`b.lineitem` (design B)	6,001,215 rows · 153,078,795 · 229,577,310,901.20	identical
`d.lineitem` (design D)	6,001,215 rows · 153,078,795 · 229,577,310,901.20	identical
`orders`	1,500,000 rows	identical
`part`	200,000 rows	identical

Sums are `sum(l_quantity)` and `sum(l_extendedprice)`. The values were the same in two independent loads.

What this means:

- **Same input.** Both engines read the same generated files, and the exact decimal types (`NUMERIC(15,2)` , `Decimal(15,2)`) preserved every cent.
- **Design D changes only the layout.** It is a re-sorted copy of B with identical content, so any B vs D difference can only come from physical order and data skipping, as Section 3.2 of the report requires.

- **Loading details that matter for reproducibility:** DBGen ends each line with `|`, which was stripped before loading; ClickHouse's default trimming of whitespace in CSV input was disabled, so both engines store the comment texts byte for byte; only the three tables used by the queries (LINEITEM, ORDERS, PART) were loaded.

Design D is physically sorted. PostgreSQL's statistics give a correlation between physical row order and `l_shipdate` of 1.000 for design D and 0.002 for design B. Design D is perfectly ordered on ship date, and design B has no order on it, as Section 3.2 of the report requires.

Query validation: all four queries agree

After one fix, all four queries return the TPC-H reference answer at SF 1 on both engines, and the engines agree with each other.

Query	TPC-H reference	PostgreSQL	ClickHouse	Verdict
Q1	4 groups, e.g. A/F sum_qty 37734107.00, count 1478493	sums and counts identical; averages agree to about 15 digits	same	Pass
Q6	123141078.23	123141078.2283	123141078.2283	Pass
Q12	MAIL 6202 / 9324, SHIP 6200 / 9262	identical	identical	Pass
Q14	16.38	16.3807786263955401	16.3807786263955401 (after fix)	Pass

Remaining visible differences are formatting only: trailing zeros, digits displayed, and ClickHouse's averages being floating-point while PostgreSQL's are exact decimals.

The Q14 finding. ClickHouse first returned 16.3807. Dividing two `Decimal` values in ClickHouse keeps the scale of the left operand, here 4 decimals, so the result was truncated, not rounded (the true value rounds to 16.3808). The relative difference from PostgreSQL, about $5 \cdot 10^{-6}$, exceeded the report's 10^{-6} tolerance. Casting the left operand to `toDecimal128(100, 12)` fixed it. The cast applies only to the final aggregated values, after the scan and join, so it should not affect execution time.

This is the "unvalidated results" pitfall of Raasveldt et al. [7], caught by the protocol before any timing. Without the check, a speed comparison on Q14 would have compared two slightly different computations.

Preliminary ingestion times (not results)

ClickHouse loaded and post-processed SF 1 faster than PostgreSQL in both manual runs, and the two runs agreed within about 10%. These are sanity checks, not reportable measurements.

Step (SF 1)	PostgreSQL run 1 / run 2 (s)	ClickHouse run 1 / run 2 (s)
Load the three tables	18.4 / 17.6	9.0 / 9.1
Post-processing: keys + `VACUUM ANALYZE` / `OPTIMIZE FINAL`	20.8 / 18.4	3.9 / 3.3
Build design D	26.4 / 29.0	15.4 / 16.2

Why these numbers cannot be used as results yet:

- **Two runs only**, timed by the shell, with no control of caches or background work.
- **Different load paths.** PostgreSQL reads the file itself (server-side `COPY`); ClickHouse receives it through `docker exec` and its client. The two times do not measure the same work.
- **Not the research question.** Ingestion time is a secondary metric (report Section 5.4), to be recorded consistently by the loading script.

What they do show: the setup is stable enough between runs to proceed, and loading SF 10 should take minutes, not hours.

Storage and data read at SF 1

ClickHouse stores LINEITEM about 2.6 times smaller than PostgreSQL's heap, and reads 8 to 12 times less data per query, because it reads only the columns a query references.

Table (design B)	PostgreSQL heap	PostgreSQL with indexes	ClickHouse compressed	ClickHouse uncompressed
LINEITEM	771 MB	899 MB	299 MiB	794 MiB
ORDERS	197 MB	229 MB	65 MiB	176 MiB
PART	29 MB	34 MB	11 MiB	31 MiB

The container's storage counter in cold runs matches these sizes, which validates both the counter and the cache reset under WSL2:

Cold run	PostgreSQL read from storage	ClickHouse read from storage	ClickHouse expected (referenced LINEITEM columns, compressed)
Q1 (7 columns)	939 MB	98 MB	94 MiB (98 MB)
Q6 (4 columns)	939 MB	76 MB	73 MiB (76 MB)
Q12 (5 columns + ORDERS)	1,180 MB	79 MB	63 MiB + ORDERS columns
Q14 (4 columns + PART)	970 MB	85 MB	79 MiB + PART columns

In every warm run, both engines read 0 bytes from storage: at SF 1, both are in the in-memory regime, as the report predicted.

Two further observations:

- **Correction for Section 3.4.** The report estimates the PostgreSQL LINEITEM heap at about 9 GB at SF 10. The measured heap is 771 MB at SF 1, so about 7.7 GB at SF 10; the 9 GB figure included the primary-key index. This is still above the roughly 6 GB of page cache left inside the 8 GB container after 2 GB of shared buffers, so H2b remains testable, with a smaller margin.
- **Sort order affects compression.** In design B, ClickHouse's date columns and `l_partkey` barely compress (`l_shipdate`: 11.49 MiB compressed, 11.45 MiB uncompressed), because the rows are ordered by order key. Design D's LINEITEM is slightly larger overall (304 vs 299 MiB) because sorting by ship date makes other columns compress less well.

Query performance at SF 1

ClickHouse is faster on all four queries, by 3.8 to 6.7 times in warm runs and 6.4 to 7.9 times in cold runs. Values are medians of 30 warm runs or 5 cold runs per engine (tag `final`), design B; the speedup S is the PostgreSQL median divided by the ClickHouse median, with a 95% bootstrap confidence interval.

Query	PostgreSQL warm (s)	ClickHouse warm (s)	S warm [95% CI]	PostgreSQL cold (s)	ClickHouse cold (s)	S cold [95% CI]
Q1	2.018	0.303	6.65 [6.44, 7.29]	2.572	0.357	7.21 [5.00, 8.52]
Q6	0.332	0.071	4.68 [3.73, 5.17]	0.743	0.112	6.62 [3.67, 9.03]

Query	PostgreSQL warm (s)	ClickHouse warm (s)	S warm [95% CI]	PostgreSQL cold (s)	ClickHouse cold (s)	S cold [95% CI]
Q12	0.497	0.122	4.05 [3.73, 4.52]	0.994	0.156	6.38 [5.58, 9.18]
Q14	0.336	0.088	3.83 [3.39, 4.33]	0.794	0.100	7.91 [6.20, 9.92]

What stands out:

- **Q1 has the largest warm speedup, contrary to the report's expectation.** Section 6.1 expected the largest speedups for Q6 and Q14, which read fewer columns, and a smaller one for Q1. PostgreSQL takes 2.0 s on Q1 but 0.33 s on Q6, although both scan the same heap; Q1 computes eight aggregates over `NUMERIC` values. This points to the type confound added to Section 6.2: PostgreSQL's software decimal arithmetic. A diagnostic run with floating-point types would test it.
- **Cold speedups are larger than warm speedups** for Q6, Q12 and Q14. When both engines must read from storage, PostgreSQL reads about 10 times more data, so the gap widens. This is the mechanism H2b relies on at SF 10, seen here in its extreme form.
- **Variability stays high on this host.** Even with 30 runs, the relative interquartile range is 11–36% for ClickHouse and 13–27% for PostgreSQL in warm runs. ClickHouse queries take only 20–300 ms, so small disturbances from Windows, WSL2 and the HTTP client weigh a lot. The confidence intervals remain narrow enough to separate S from 1, but they will limit how small a change in S can be detected between sizes.

Design D: data skipping

Sorting LINEITEM by ship date speeds up Q6 and Q14 by 2.7–4.6 times in both engines, as expected, but it also changes Q12 in ClickHouse, which the report did not anticipate. Warm medians, 30 runs each:

Query	PostgreSQL B → D (s)	ClickHouse B → D (s)	S under B → D	ClickHouse LINEITEM rows read under D	Report's a(q, D)
Q1	2.018 → 2.238	0.303 → 0.263	6.65 → 8.49	98.7%	≈ 0.98
Q6	0.332 → 0.121	0.071 → 0.019	4.68 → 6.45	15.3%	≈ 1/7 (14%)
Q12	0.497 → 0.602	0.122 → 0.053	4.05 → 11.35	43%	1 (no skipping)
Q14	0.336 → 0.073	0.088 → 0.023	3.83 → 3.15	1.4%	≈ 1/80 (1.25%)

Rows read come from ClickHouse's query log, after subtracting the joined table (ORDERS 1.5 M rows, PART 0.2 M). For Q1, Q6 and Q14 they match the fractions the report computed from the query text, which validates the a(q, P) definition.

- **Q12 in ClickHouse.** Q12 filters on receipt date, not ship date, yet ClickHouse read only 43% of LINEITEM. Its predicates imply ship date < commit date < receipt date < 1995-01-01, so ship date < 1995-01-01, which holds for about 3/7 of the rows. The result is consistent with ClickHouse deriving this bound from the chain of comparisons; PostgreSQL did not. Under design D, Q12 is therefore a data-skipping query for ClickHouse only, and the report's Table 4 should list a(Q12, D) ≈ 0.43, with a note that only one engine exploits it.
- **PostgreSQL gets slower on Q1 (+11%) and Q12 (+21%) under D.** Neither query can skip with the BRIN index. One possible cause for Q12 is that LINEITEM sorted by ship date no longer follows the order-key order of ORDERS, which may hurt the hash join. This is not yet verified; the saved plans in `results/plans/` should be compared.
- **The gap narrows only for Q14.** Where both engines skip comparably (Q14), S falls from 3.83 to 3.15. For Q6, ClickHouse gains more than PostgreSQL, and S rises.

What SF 1 says about the hypotheses

SF 1 supports H1 at one of four sizes and says nothing yet about H2a or H2b, which concern how the speedup changes between sizes.

Hypothesis	Decision rule (report Table 6)	Status after SF 1
H1: ClickHouse faster at every size	CI of S above 1 at all four sizes	CI above 1 for all four queries at SF 1; partial, 3 sizes remaining

Hypothesis	Decision rule (report Table 6)	Status after SF 1
H2a: S roughly constant while both engines fit in memory	CI of r_1, r_2 within [0.9, 1.1]	Not testable yet: needs SF 2 and 4. SF 1 confirms both engines are in memory
H2b: S rises when only PostgreSQL leaves memory	CI of r_3 above 1.1, with the expected memory regimes	Not testable yet: needs SF 4 and 10

Two results already bear on the later tests. The higher cold speedups show that storage reads widen the gap, which is the mechanism behind H2b. And the high variability on this host means that a change in S of about 10%, the width of the H2a band, may be hard to resolve: confidence intervals on r will be roughly as wide as the band.

Implications for the report

The setup work revises four sections of the M1 report, mostly Section 5.3 (environment), and the measurements add six more revisions, listed after the first table. None of the changes affects the research question, the hypotheses or the decision rules. They are the "protocol revisions" that Milestone 2 asks for.

Report section	What changes	Why
5.3 Environment	Host is a Windows laptop with WSL2, not native Linux; Docker Engine inside Ubuntu; 12 GB given to WSL2, swap off, cgroup v2 forced	No native Linux machine was available; this configuration keeps the 8 GB container budget and the cgroup counters the protocol relies on
5.3 Versions	Replace "18.X" and "26.8.X" with PostgreSQL 18.6 and ClickHouse 26.8.36.3, plus their image digests	The versions are now pinned
5.3 CPU pinning	State that `cpuset 0-3` is two physical cores with their hyperthreads	Recorded from `lscpu -e`; under WSL2 the mapping is decided by Windows, so it is approximate

Report section	What changes	Why
5.1 Loading	Add: trailing delimiter stripped; ClickHouse CSV whitespace trimming disabled; only LINEITEM, ORDERS and PART loaded; plain `String` in ClickHouse (no `LowCardinality`)	Makes the load reproducible and keeps the data byte-identical in both engines
5.2 Workload	Replace "Only the syntax required by each SQL dialect is changed" with: date-literal syntax plus explicit decimal casts in ClickHouse Q6 and Q14, which change types but not meaning; joins written as LINEITEM JOIN the smaller table	Q14 failed validation without the cast; ClickHouse builds its hash table from the right-hand table
5.2 Validation	Report the SF 1 outcome: all four queries match the reference and agree between engines; describe the Q14 truncation and its fix	Evidence for O2
6.2 Threats	Add WSL2 virtualization, laptop turbo and thermal behaviour, and the open question of whether Windows caches the virtual disk during cold runs	New threats introduced by the host change

Suggested wording for the new environment paragraph in Section 5.3:

> The experiments run on a laptop with an Intel Core i5-10300H (4 cores, 8 threads), 16 GB of RAM and an SSD, under Windows with WSL2 (Ubuntu 22.04, cgroup v2, systemd). WSL2 is given 12 GB of memory and no swap. Docker Engine runs inside the Linux system rather than through Docker Desktop, so that the containers' cgroup counters are directly readable and the page cache can be cleared. Each engine runs in its own container pinned to the same four logical CPUs with an 8 GB memory limit; only one container runs during measurements. Native Linux was not available; the threats this introduces are discussed in Section 6.2.

Table 5 needs no change: every value was verified in the running engines.

Revisions from the pilot and the SF 1 measurements:

Report section	What changes	Evidence
3.4 Rationale, Table 3	PostgreSQL LINEITEM heap at SF 10 ≈ 7.7 GB, not 9 GB; replace Table 3′s estimates by measured sizes	Heap 771 MB at SF 1; the 899 MB total includes the primary-key index
4 Table 4	Q12 under design D: a ≈ 0.43 for ClickHouse, which derives an implied ship-date bound; PostgreSQL does not skip	ClickHouse read 43% of LINEITEM rows for Q12 under D
5.4 Procedure	30 warm runs per query by default, not 10	Pilot relative IQR 20–69% for ClickHouse with 3 runs; report′s own rule (above 10% → 30 runs)
5.4 Procedure	Cold runs are valid under WSL2	Cold-run storage reads match table and column sizes; warm runs read 0 bytes
6.1 Expected outcomes	Q1 shows the largest speedup at SF 1, not Q6 or Q14; discuss the decimal-arithmetic explanation	S = 6.65 for Q1 vs 3.83–4.68 for the others
6.2 Threats	Host variability: warm relative IQR 11–36% even with 30 runs; limits the precision of r	Final SF 1 run